

第 3A 卷 贪心与 DP 辨析

考试速查：主查贪心能不能用：看是否有交换论证、排序依据、堆维护、区间选择；一旦局部选择影响未来就转 DP 或搜索。

Contents

第 3A 卷 贪心与 DP 辨析	1
考试速查定位	1
GREEDY-00 贪心路由与短模板	2
GREEDY-01: 什么时候贪心，什么时候 DP	6
1. 一句话判别	7
2. 贪心 vs DP 判别表	7
3. 贪心常用证明套路	7
4. 四个必须记住的反例	8
5. 考场决策流程	8
6. 小数据暴力验证贪心	8
7. 典型替代路线	9
GREEDY-02: 常见贪心模型补充	9
1. 区间覆盖整段	11
2. 双指针配对：最少船	11
3. 二分答案 + 贪心 check	11
4. 截止时间收益最大：DSU 找空位	12
5. 相邻交换排序思路	13
v0.2 本卷例题训练区	14
V03A-EX01 最少总等待时间排序贪心	14
V03A-EX02 最多不重叠活动	15
V03A-EX03 最少点覆盖闭区间	16
V03A-EX04 合并果子的堆贪心	17
V03A-EX05 最少会议室	18
V03A-EX06 截止时间课程的反悔贪心	19
V03A-EX07 0/1 背包反例转 DP	21
V03A-EX08 普通硬币找零反例转完全背包	22
V03A-EX09 加权区间选择转 DP	23
V03A-EX10 相邻限制选择转线性 DP	24
V03A-CEX01 活动选择结束时间贪心	25
V03A-CEX02 会议室最少数量	25
V03A-CEX03 最少点刺破区间	26
V03A-CEX04 单价贪心失败的 0/1 背包	27
V03A-CEX05 截止日任务反悔贪心	28

第 3A 卷 贪心与 DP 辨析

考试速查定位

第 3A 卷 贪心与 DP 辨析

- 这是第几卷：03A。
- 主要查什么：主查贪心能不能用：看是否有交换论证、排序依据、堆维护、区间选择；一旦局部选择影响未来就转 DP 或搜索。
- 什么时候翻：不知道能不能贪心、需要证明交换/反例、贪心失败要转 DP 时翻。
- 题面关键词：排序贪心；区间；堆；局部最优；反例；DP 对照。

自动由 03_modules/GREEDY-*.md 重建。定位是帮助初学者判断什么时候能贪心，什么时候必须回到 DP/记忆化搜索。

GREEDY-00 贪心路由与短模板

模块编号: GREEDY-00

模块名称: 贪心路由与短模板

标签: 贪心、区间选点、活动选择、排序贪心、堆贪心、反悔贪心

一句话用途: 看到“每次选当前最优”或“排序后一扫”时, 先用本表判断是哪类贪心, 再抄最短模板。

题面触发词:

- 最少点覆盖所有区间、最多不重叠活动。
- 按截止时间、结束时间、代价、收益排序。
- 每次取最小、每次取最大、合并代价最小。
- 先选进去, 超过限制再删掉一个最差的。

什么时候用:

- 局部选择可以通过交换论证变成全局最优。
- 题面有明确的“最早结束”“当前最小”“保留最好若干个”等信号。
- 数据范围要求 $O(n \log n)$ 或 $O(n)$, 暴力枚举会超时。
- 可以把所有对象先排序, 再从左到右扫描。

不要什么时候用:

- 当前选择会影响未来状态, 且没有明显可交换性。
- 需要记录多个维度历史状态, 通常是 DP。
- 图上最短路、连通性、树形依赖不要硬套普通贪心。
- 只凭样例猜规则, 不能解释为什么删这个、选这个。

复杂度:

- 排序贪心: $O(n \log n)$ 。
- 区间选点/活动选择: 排序 $O(n \log n)$, 扫描 $O(n)$ 。
- 堆贪心: $O(n \log n)$ 。
- 反悔贪心: 排序加堆, $O(n \log n)$ 。

数据范围参考:

- $n \leq 2e5$: 排序、堆贪心常用。
- $n \leq 1e6$: 能线性更好, 排序也可能可过但注意常数。
- $n \leq 20$: 可先暴力枚举验证贪心猜想。

依赖的标准容器:

- `vector<Seg>`: 区间, 默认 1-index。
- `vector<Item>`: 多字段任务。
- `priority_queue`: 堆贪心和反悔贪心。

输入如何整理:

```
struct Seg {
    int l;
    int r;
    int id;
};

int n;
cin >> n;
vector<Seg> seg(n + 1);
for (int i = 1; i <= n; i++) {
    cin >> seg[i].l >> seg[i].r;
    seg[i].id = i;
}
```

整理顺序:

1. 先把对象变成 struct, 保留 id。
2. 明确排序关键字: 结束时间、开始时间、截止时间、收益、代价。

3. 扫描时维护当前答案、当前资源或堆。
4. 遇到超限时，反悔贪心从堆里删掉最差选择。

接口：

choose_points(seg) -> 闭区间最少选点覆盖，每次选当前右端点。
 activity_select(seg) -> 半开区间 $[l, r)$ 或允许端点相接时的最多互不冲突活动。
 min_merge_cost(a) -> 每次合并最小两项，Huffman/合并果子。
 min_rooms(seg) -> 半开区间 $[l, r)$ 或结束即释放资源时的最少会议室/机器数。
 regret_max_count(course) -> 截止时间内最多完成任务，超时删耗时最大。

输出能力：

- 输出最少点数、最多活动数、最小合并代价。
- 输出所选点、所选活动编号。
- 输出最少资源数或最多可完成任务数。

下游可接：

- CPP-003 排序与比较函数。
- CPP-004 优先队列。
- BRUTE 暴力枚举验证。
- 二分答案中的 check。
- GREEDY-01 贪心与 DP 判别卡：证明不了贪心时，回到 DFS/DP。

可拼接模块：

题面信号	路由	核心选择
最少点覆盖所有闭区间	区间选点	按右端点升序，没覆盖就选右端点
最多不重叠活动	活动选择	按结束时间升序，能接就选
排队等待、按代价处理	排序贪心	按题面最紧约束排序后扫描
合并果子、每次合并代价	堆贪心	每次取两个最小
最少会议室/机器	堆贪心	小根堆维护当前结束时间
截止时间内最多任务	反悔贪心	按截止时间扫，超时删耗时最大
最多收益且数量/时间受限	反悔贪心	先放入堆，超限删最差收益或最大耗时

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Seg {
    int l;
    int r;
    int id;
};

struct Course {
    int t;
    int d;
    int id;
};

bool by_right(const Seg &a, const Seg &b) {
    if (a.r != b.r) return a.r < b.r;
    return a.l < b.l;
}

vector<int> choose_points(vector<Seg> seg) {
    int n = (int)seg.size() - 1;
    sort(seg.begin() + 1, seg.end(), by_right);

    vector<int> ans;
    int last = numeric_limits<int>::min();
```

```

    for (int i = 1; i <= n; i++) {
        if (seg[i].l > last) {
            last = seg[i].r;
            ans.push_back(last);
        }
    }
    return ans;
}

int activity_select(vector<Seg> seg) {
    int n = (int)seg.size() - 1;
    sort(seg.begin() + 1, seg.end(), by_right);

    int cnt = 0;
    int last_end = numeric_limits<int>::min();
    for (int i = 1; i <= n; i++) {
        if (seg[i].l >= last_end) {
            cnt++;
            last_end = seg[i].r;
        }
    }
    return cnt;
}

ll min_merge_cost(const vector<ll> &a) {
    int n = (int)a.size() - 1;
    priority_queue<ll, vector<ll>, greater<ll>> pq;
    for (int i = 1; i <= n; i++) {
        pq.push(a[i]);
    }

    ll ans = 0;
    while (pq.size() > 1) {
        ll x = pq.top();
        pq.pop();
        ll y = pq.top();
        pq.pop();
        ans += x + y;
        pq.push(x + y);
    }
    return ans;
}

int min_rooms(vector<Seg> seg) {
    int n = (int)seg.size() - 1;
    sort(seg.begin() + 1, seg.end(), [](const Seg &a, const Seg &b) {
        if (a.l != b.l) return a.l < b.l;
        return a.r < b.r;
    });

    priority_queue<int, vector<int>, greater<int>> ends;
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        // 半开区间  $[L, r)$  可用  $<=$ ; 闭区间  $[L, r]$  且端点相同算冲突时改成  $<$ 。
        while (!ends.empty() && ends.top() <= seg[i].l) {
            ends.pop();
        }
        ends.push(seg[i].r);
        ans = max(ans, (int)ends.size());
    }
    return ans;
}

```

```

int regret_max_count(vector<Course> c) {
    int n = (int)c.size() - 1;
    sort(c.begin() + 1, c.end(), [](const Course &a, const Course &b) {
        if (a.d != b.d) return a.d < b.d;
        return a.t < b.t;
    });

    priority_queue<int> chosen_time;
    ll used = 0;
    for (int i = 1; i <= n; i++) {
        used += c[i].t;
        chosen_time.push(c[i].t);
        if (used > c[i].d) {
            used -= chosen_time.top();
            chosen_time.pop();
        }
    }
    return (int)chosen_time.size();
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Seg> seg(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> seg[i].l >> seg[i].r;
        seg[i].id = i;
    }

    cout << choose_points(seg).size() << '\n';
    cout << activity_select(seg) << '\n';

    return 0;
}

```

调用示例:

```

// 区间选点: 输出最少点数和点的位置。
vector<int> pts = choose_points(seg);
cout << pts.size() << '\n';
for (int x : pts) cout << x << ' ';
cout << '\n';

// 合并果子。
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
cout << min_merge_cost(a) << '\n';

// 反悔贪心: 课程 t 为耗时, d 为截止时间。
vector<Course> c(n + 1);
for (int i = 1; i <= n; i++) {
    cin >> c[i].t >> c[i].d;
    c[i].id = i;
}
cout << regret_max_count(c) << '\n';

```

常见坑:

- 区间选点是闭区间覆盖, 判断通常是 $l > \text{last}$ 才需要新点。
- 活动选择是否允许首尾相接要看题面; 允许用 $l \geq \text{last_end}$, 不允许改成 $l > \text{last_end}$ 。
- 排序比较函数不能写 $\leq$ 。
- 贪心排序关键字错了很难从样例看出, 至少用小数据暴力对拍。

- 堆贪心默认 `priority_queue` 是大根堆，小根堆要写 `greater<T>`。
- 反悔贪心的堆里存“将来要删的最差项”：超时删最大耗时，保收益删最小收益。
- 区间端点可能到 $1e9$ ，端点用 `int` 多数够；代价、答案用 `long long`。
- 无法说出交换论证、领先性或反悔不变量时，不要硬套贪心，先用小数据暴力或 DP-25 记忆化保底。

暴力/部分替代：

- $n \leq 20$ ：枚举子集，检查覆盖或互不冲突，验证贪心。
- 合并果子小数据：每次线性找两个最小， $O(n^2)$ 。
- 会议室小数据：按时间点或每个活动扫描已有房间。
- 截止时间任务小数据：全排列或子集枚举，检查能否按截止时间完成。

升级方向：

枚举所有选法 -> 按右端点排序的区间贪心

每次线性找最小 -> `priority_queue`

排序后超限失败 -> 加堆做反悔

贪心无法证明 -> 改 DFS/DP，先拿小数据

最小测试样例：

输入

```
3
1 3
2 5
6 7
```

输出

```
2
2
```

补充自测：

区间选点：[1,3],[2,5],[6,7] -> 选点 3,7

活动选择：[1,3],[2,5],[6,7] -> 最多 2 个

合并果子：1 2 3 4 -> 19

反悔任务：(3,5),(4,6),(2,6) -> 最多 2 个

GREEDY-01：什么时候贪心，什么时候 DP

模块编号：GREEDY-01

模块名称：贪心与 DP 判别卡

标签：贪心、DP、反例、交换论证、局部最优、模型判别

一句话用途：防止考场上“看到最大/最小就猜贪心”，用一张判别表决定是写贪心、写 DP，还是先暴力对拍。

题面触发词：

- “每次选择当前最小/最大”
- “排序后从左到右”
- “截止时间、结束时间、覆盖区间”
- “容量/预算/选或不选”
- “当前选择会影响后续状态”
- “求最大收益/最小代价”

什么时候用：

- 你怀疑题目可以贪心，但需要快速判断是否可靠。
- 同一个题面既像排序贪心，又像 DP。
- 需要给自己一个反例检查流程，避免样例过了但正式数据错。

不要什么时候用：

- 已经是标准最短路、DSU、线段树、背包、LCS 等明确模型时，不要先纠结贪心证明。
- 题目只要求模拟规则，不涉及最优选择。

复杂度：

- 贪心通常 $O(n \log n)$ 或 $O(n)$ 。
- DP 通常是 “状态数 $\times$ 转移数”。
- 暴力对拍小数据常用 $O(2^n)$ 、 $O(n!)$ 。

数据范围信号：

- $n \leq 20$ ：先暴力枚举，验证贪心猜想。
- $n \leq 5000$ ： $O(n^2)$ DP 常可拿分。
- $n \leq 2e5$ ：若 DP 状态太大，优先考虑贪心、数据结构优化或二分答案。

依赖的标准容器：

- vector
- sort
- priority_queue
- map/unordered_map 用于记忆化或对拍记录。

输入如何整理：

- 先把对象整理成 struct Item { ... }。
- 写出 “候选贪心关键字”：右端点、截止时间、单位收益、耗时、收益。
- 写出 “如果用 DP，需要记什么状态”：位置、容量、上一个选择、已用时间、集合。

接口：

题面 $\rightarrow$ 判别表 $\rightarrow$ 贪心证明/反例检查 $\rightarrow$ 贪心模板或 DP 模型

输出能力：

- 决定优先写贪心还是 DP。
- 给出贪心常用证明套路。
- 给出典型反例，防止误用。

下游可接：

- GREEDY-00 贪心路由与短模板
- DP-01 DP 路由表
- BRUTE 枚举与对拍

可拼接模块：

- Sorting：排序贪心。
- PriorityQueue：堆贪心、反悔贪心。
- Knapsack/LinearDP：贪心失败时的 DP 替代。

1. 一句话判别

能证明 “局部选择不吃亏” $\rightarrow$ 贪心。

必须记住容量、上一个状态、已选集合、前缀最优 $\rightarrow$ DP。

证明不了 $\rightarrow$ 先写暴力/记忆化拿分，再用小数据找反例。

2. 贪心 vs DP 判别表

题面形状	更可能是贪心	更可能是 DP
区间最多不重叠	按结束时间选	有权重收益时变成 DP
区间最少点覆盖	按右端点选点	有点权/限制选点数时变成 DP
合并代价最小	每次合并两个最小，堆	合并只能相邻时是区间 DP
任务截止时间最多完成	按截止时间 + 反悔堆	每个任务收益不同且约束复杂时 DP/费用流
背包最大价值	分数背包可贪心	0/1 背包通常 DP
硬币找零	特定币制可贪心	普通币制用完全背包
最短路	Dijkstra 是带证明的贪心	有负边不能直接贪心
每步代价相同最少步数	BFS	不是普通排序贪心
前 i 个元素最优	很少纯贪心	线性 DP
选/不选且有容量	很少纯贪心	背包 DP

3. 贪心常用证明套路

交换论证：

假设最优解没有选我的贪心选择。
把最优解里的某个选择换成贪心选择。
如果答案不变差，说明存在一个包含贪心选择的最优解。

领先性证明：

按步骤比较贪心解和任意最优解。
证明每一步后，贪心占用资源不更多，完成进度不少。

删最差反悔：

先把当前任务放进去。
如果资源超限，从已选任务中删掉最差的。
证明删掉它后，保留的集合对后续最有利。

切分/割性质：

图论里的 Kruskal、Prim、Dijkstra 都有特殊证明。
它们是安全的算法模块，不是“凭感觉每次选最小边”。

4. 四个必须记住的反例

0/1 背包不能按性价比贪心：

容量 10

物品：(w=6, v=12), (w=5, v=10), (w=5, v=10)

性价比都相近，若先选 6 只能得 12；选两个 5 得 20。

普通硬币找零不能总选最大：

硬币 1, 3, 4，目标 6

贪心：4+1+1，共 3 枚

最优：3+3，共 2 枚

有权区间选择不能只按结束时间：

区间 A=[1, 2]，收益 1

区间 B=[2, 3]，收益 1

区间 C=[1, 3]，收益 100

最多数量可贪心，最大收益要 DP。

相邻合并不能 Huffman：

合并果子可任意合并 -> 每次取两堆最小。

合并石子只能合并相邻 -> 区间 DP。

5. 考场决策流程

1. 先问：有没有容量、次数、上一个选择、集合状态？
有 -> 优先 DP/记忆化。
2. 再问：排序后每个对象只处理一次吗？
是 -> 可能贪心或扫描线。
3. 再问：我能说出“为什么这个局部选择不吃亏”吗？
能 -> 写贪心。
不能 -> 小数据暴力验证，或者写 DP 部分分。
4. 贪心猜想有两个以上排序关键字都像对的：
先不要硬冲满分，写暴力/DP 小数据版本保底。

6. 小数据暴力验证贪心

考场不能写太复杂的对拍，但可以用脑内或本地小样例检查。若允许临时调试，可写一个 $n \leq 20$ 的暴力版本。

```
int brute_best(vector<int>& w, vector<int>& v, int W) {
    int n = (int)w.size() - 1;
    int ans = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int sw = 0, sv = 0;
```



```

    for (int i = 1; i <= n; i++) {
        if (mask & (1 << (i - 1))) {
            sw += w[i];
            sv += v[i];
        }
    }
    if (sw <= W) ans = max(ans, sv);
}
return ans;
}

```

7. 典型替代路线

贪心失败信号	替代模型
区间有收益	线性 DP / 加权区间调度
容量限制且物品不可拆	0/1 背包
硬币数量不限但币制普通	完全背包
相邻合并	区间 DP
树上选择受父子影响	树形 DP
图上局部最短但边权复杂	最短路算法, 不用手写普通贪心

常见坑:

- 只凭样例确认贪心, 正式数据很容易被反例卡。
- 把“排序后扫描”当成所有最优题的万能解。
- 0/1 背包、加权区间、相邻合并是最常见的贪心误判。
- 小根堆/大根堆方向错, 会把“反悔删最差”写反。
- Dijkstra/Kruskal 虽然有贪心味道, 但应按图论模块模板写。

暴力/部分分替代:

- $n \leq 20$: 子集枚举验证最优。
- $n \leq 10$: 全排列验证顺序类问题。
- 写不出贪心证明时, 先交 DFS/记忆化/ $O(n^2)$ DP。
- 大数据不会满分时, 保留小数据精确解, 并输出合法兜底。

升级方向:

猜贪心 -> 找反例 -> 能证明则 GREEDY-00

猜贪心 -> 有容量/历史状态 -> DP-01

贪心样例过但不确定 -> 暴力小数据保底

普通贪心超限/不够 -> 堆贪心或反悔贪心

最小测试样例:

硬币:

1 3 4

target = 6

贪心最大硬币失败, 应该用完全背包最少硬币。

区间:

[1,2] value=1

[2,3] value=1

[1,3] value=100

最大收益不是活动选择贪心。

GREEDY-02: 常见贪心模型补充

模块编号: GREEDY-02

模块名称: 常见贪心模型补充

标签: 贪心、区间覆盖、双指针、二分答案、截止时间、相邻交换

一句话用途：补齐 GREEDY-00 之外的高频贪心：区间覆盖整段、排序双指针配对、二分答案中的贪心 check、截止时间收益最大。

题面触发词：

- “最少区间覆盖 $[L, R]$ ”
- “每艘船最多两人/两端配对/尽量多匹配”
- “最大化最小值/最小化最大值”
- “每个任务有截止时间和收益”
- “排序顺序会影响总代价”

什么时候用：

- 排序后只需要扫描、双指针或堆维护。
- 当前选择有明确不变量：覆盖最远、配对最省、保留收益最大。
- $\text{check}(x)$ 能用从左到右的贪心验证可行性。

不要什么时候用：

- 区间选择有收益且要最大化总收益，常转 DP。
- 配对选择需要记复杂历史，双指针不一定成立。
- 截止时间有依赖关系或任务耗时不统一，普通收益贪心可能失效。
- 不能说明为什么当前选择不吃亏时，先回 GREEDY-01 或 DP。

复杂度：

- 排序 + 扫描/双指针/堆： $O(n \log n)$ 。
- 二分答案 + 贪心检查： $O(n \log V)$ 或 $O(n \log n \log V)$ 。
- DSU 找空位： $O(n \log n + n \alpha(n))$ 。

数据范围信号：

- $n \leq 2e5$ ：排序、堆、DSU 贪心常用。
- 答案范围很大，且有单调性：二分答案 + 贪心 check。
- 小数据可先枚举验证贪心。

依赖的标准容器：

- `vector`
- `sort`
- `priority_queue`
- DSU 或简单并查集数组。

输入如何整理：

- 区间统一成 `struct Seg { int l, r; };`
- 任务统一成 `struct Job { int deadline, profit; };`
- 双指针配对先排序数组。

接口：

```
int cover_interval(Seg seg[], int n, int L, int R);
int min_boats(vector<int> weight, int limit);
long long max_profit_jobs(vector<Job> jobs);
```

输出能力：

- 最少覆盖区间数。
- 最少配对资源数。
- 最大可得收益。
- 二分答案可行性。

下游可接：

- GREEDY-00 贪心路由。
- GREEDY-01 贪心与 DP 判别。
- DSU 模块。
- 二分答案模块。

可拼接模块：

- Sorting：所有模型都先排序。
- PriorityQueue：收益/截止时间反悔。
- DSU：每个时间槽最多放一个任务。

1. 区间覆盖整段

给若干区间，问最少选几个覆盖 $[L, R]$ 。每一步在所有 $l \leq$ 当前覆盖右端 $+ 1$ 的区间里，选 r 最远的那个。

```
#include <bits/stdc++.h>
using namespace std;

struct Seg {
    int l;
    int r;
};

const int MAXN = 200000 + 5;

int cover_interval(Seg seg[], int n, int L, int R) {
    sort(seg + 1, seg + n + 1, [](const Seg& a, const Seg& b) {
        if (a.l != b.l) return a.l < b.l;
        return a.r > b.r;
    });

    int i = 1;
    long long cur = (long long)L - 1;
    int ans = 0;

    while (cur < R) {
        long long best = cur;
        while (i <= n && seg[i].l <= cur + 1) {
            best = max(best, (long long)seg[i].r);
            i++;
        }
        if (best == cur) return -1;
        cur = best;
        ans++;
    }
    return ans;
}
```

这个模板默认整数闭区间 $[L, R]$ 。如果区间是实数或闭开边界， $cur + 1$ 要按题意改成 $seg[i].l \leq cur$ 。

2. 双指针配对：最少船

每艘船最多两人，重量和不超过 $limit$ ，求最少船数。

```
int min_boats(int weight[], int n, int limit) {
    sort(weight + 1, weight + n + 1);
    int l = 1;
    int r = n;
    int ans = 0;

    while (l <= r) {
        if ((long long)weight[l] + weight[r] <= limit) {
            l++;
            r--;
        } else {
            r--;
        }
        ans++;
    }
    return ans;
}
```

不变量：最重的人必须上船；如果能和最轻的人配，就配掉最轻的，否则最重的人只能单独走。

3. 二分答案 + 贪心 check

最大化最小距离：给若干位置，选 k 个点，使相邻距离至少 $dist$ 是否可行。

```

bool can_pick_with_distance(int x[], int n, int k, int dist) {
    int cnt = 1;
    int last = x[1];
    for (int i = 2; i <= n; i++) {
        if (x[i] - last >= dist) {
            cnt++;
            last = x[i];
        }
    }
    return cnt >= k;
}

int maximize_min_distance(int x[], int n, int k) {
    sort(x + 1, x + n + 1);
    int lo = 0;
    int hi = x[n] - x[1];
    while (lo < hi) {
        int mid = (lo + hi + 1) / 2;
        if (can_pick_with_distance(x, n, k, mid)) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}

```

4. 截止时间收益最大: DSU 找空位

每个任务耗时 1 天, 有截止时间和收益, 每天最多做一个任务。按收益从大到小, 尽量塞到不超过 deadline 的最晚空位。

```

struct Job {
    int deadline;
    int profit;
};

struct SlotDSU {
    vector<int> fa;

    void init(int n) {
        fa.resize(n + 1);
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x) {
        while (x != fa[x]) {
            fa[x] = fa[fa[x]];
            x = fa[x];
        }
        return x;
    }

    bool occupy(int x) {
        int p = find(x);
        if (p == 0) return false;
        fa[p] = find(p - 1);
        return true;
    }
};

long long max_profit_jobs(vector<Job> jobs) {
    int max_deadline = 0;
    for (auto job : jobs) max_deadline = max(max_deadline, job.deadline);
    int limit = min(max_deadline, (int)jobs.size());

    sort(jobs.begin(), jobs.end(), [](const Job& a, const Job& b) {
        if (a.profit != b.profit) return a.profit > b.profit;
    });
}

```

```

        return a.deadline < b.deadline;
    });

    SlotDSU dsu;
    dsu.init(limit);

    long long ans = 0;
    for (auto job : jobs) {
        if (job.deadline <= 0 || job.profit <= 0) continue;
        int slot = min(job.deadline, limit);
        if (dsu.occupy(slot)) ans += job.profit;
    }
    return ans;
}

```

5. 相邻交换排序思路

当题目问“怎样排序使总代价最小”，常用相邻交换。比较相邻两个元素 a, b ，看顺序 a, b 和 b, a 哪个总代价小，再推出排序比较函数。

示例：若代价是等待时间总和，短任务优先通常更好。

```

long long min_total_waiting_time(vector<int> t) {
    sort(t.begin(), t.end());
    long long elapsed = 0;
    long long ans = 0;
    for (int x : t) {
        ans += elapsed;
        elapsed += x;
    }
    return ans;
}

```

常见坑：

- 区间覆盖和区间选点不是一个模型：覆盖整段选“最远右端”，选点覆盖区间选“当前右端点”。
- 双指针配对必须先排序。
- 二分答案的 check 必须有单调性：可行后更宽松也可行，或不可行后更严格也可行。
- 截止时间收益最大里要放到“不超过 deadline 的最晚空位”，不是最早空位。
- 相邻交换推出的比较函数不能写 $<=$ 。

暴力/部分分替代：

- $n \leq 20$ ：子集枚举检查区间覆盖或任务收益。
- 配对题小数据可 DFS 枚举配对。
- 二分答案不会写时，先枚举小答案或写贪心构造。
- 截止时间任务小数据可枚举任务子集，再按 deadline 排序验证。

升级方向：

区间覆盖小暴力 -> 按左端排序 + 选最远右端
 配对暴力 -> 排序双指针
 枚举答案 -> 二分答案 + 贪心 check
 收益任务枚举 -> 收益排序 + DSU 找空位
 排序猜想 -> 相邻交换推出比较函数

最小测试样例：

区间覆盖 [1,10]:
 [1,3] [2,6] [4,10]
 输出: 3

最少船：

weights = 3 2 2 1, limit = 3
 输出: 3

截止时间收益：

(deadline=1, profit=10), (deadline=1, profit=20), (deadline=2, profit=5)
 输出: 25

v0.2 本卷例题训练区

这一节是 0.2 新增的实战例题。每题都配完整可运行代码和样例；考试时优先看“覆盖模块”和“考场用途”，再复制对应代码骨架。

V03A-EX01 最少总等待时间排序贪心

- 归属卷：第 3A 卷
- 覆盖模块：排序贪心、交换论证、最短处理时间优先
- 考场用途：任务顺序可自由调整且目标是等待时间总和时，按耗时从小到大排序。

题目描述：有 n 个任务，第 i 个任务耗时 $t[i]$ 。一次只能处理一个任务，任务顺序可以任意调整。一个任务的等待时间是它开始前已经处理完的任务总耗时。求最小总等待时间。

输入格式：第一行一个整数 n 。第二行 n 个整数表示任务耗时。

输出格式：输出最小总等待时间。

样例输入：

```
3
3 1 2
```

样例输出：

```
4
```

完整代码：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> t(n + 1);
    for (int i = 1; i <= n; i++) cin >> t[i];
    sort(t.begin() + 1, t.end());

    long long elapsed = 0;
    long long ans = 0;
    for (int i = 1; i <= n; i++) {
        ans += elapsed;
        elapsed += t[i];
    }

    cout << ans << '\n';
    return 0;
}
```

测试设计：

测试 1 输入：

```
3
3 1 2
```

期望输出：

```
4
```

测试 2 输入：

```
4
5 5 1 2
```

期望输出：

```
12
```

V03A-EX02 最多不重叠活动

- 归属卷：第 3A 卷
- 覆盖模块：区间贪心、按右端点排序、活动选择
- 考场用途：区间互不重叠、要选最多个时，优先尝试按结束时间从早到晚选。

题目描述：给定 n 个半开区间 $[l, r)$ ，选择尽量多的区间，使任意两个被选区间不重叠。若一个区间的结束时间等于另一个区间的开始时间，允许同时选择。

输入格式：第一行一个整数 n 。接下来 n 行，每行两个整数 l r 。

输出格式：输出最多能选择的区间数量。

样例输入：

```
4
1 3
2 4
3 5
0 7
```

样例输出：

```
2
```

完整代码：

```
#include <bits/stdc++.h>
using namespace std;

struct Seg {
    long long l;
    long long r;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Seg> seg(n + 1);
    for (int i = 1; i <= n; i++) cin >> seg[i].l >> seg[i].r;

    sort(seg.begin() + 1, seg.end(), [](const Seg &a, const Seg &b) {
        if (a.r != b.r) return a.r < b.r;
        return a.l < b.l;
    });

    long long lastEnd = -(1LL << 60);
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        if (seg[i].l >= lastEnd) {
            ans++;
            lastEnd = seg[i].r;
        }
    }

    cout << ans << '\n';
    return 0;
}
```

测试设计：

测试 1 输入：

```
4
1 3
2 4
```

```
3 5
0 7
```

期望输出:

```
2
```

测试 2 输入:

```
5
1 2
2 3
3 4
1 4
4 5
```

期望输出:

```
4
```

V03A-EX03 最少点覆盖闭区间

- 归属卷: 第 3A 卷
- 覆盖模块: 区间选点、排序贪心、右端点策略
- 考场用途: 每个闭区间至少被一个点覆盖时, 每次在当前最早结束区间的右端点放点。

题目描述: 给定 n 个闭区间 $[l, r]$ 。请选择尽量少的整数点, 使每个区间内至少包含一个被选点。输出最少点数和这些点。

输入格式: 第一行一个整数 n 。接下来 n 行, 每行两个整数 l r 。

输出格式: 第一行输出最少点数。第二行按选择顺序输出所有点。

样例输入:

```
3
1 3
2 5
3 6
```

样例输出:

```
1
3
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

struct Seg {
    long long l;
    long long r;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Seg> seg(n + 1);
    for (int i = 1; i <= n; i++) cin >> seg[i].l >> seg[i].r;

    sort(seg.begin() + 1, seg.end(), [](const Seg &a, const Seg &b) {
        if (a.r != b.r) return a.r < b.r;
        return a.l < b.l;
    });

    vector<long long> points;
    long long lastPoint = -(1LL << 60);
    for (int i = 1; i <= n; i++) {
```



```

        if (lastPoint < seg[i].l) {
            lastPoint = seg[i].r;
            points.push_back(lastPoint);
        }
    }

    cout << points.size() << '\n';
    for (int i = 0; i < (int)points.size(); i++) {
        if (i) cout << ' ';
        cout << points[i];
    }
    cout << '\n';
    return 0;
}

```

测试设计:

测试 1 输入:

```

3
1 3
2 5
3 6

```

期望输出:

```

1
3

```

测试 2 输入:

```

4
1 2
3 4
2 3
5 5

```

期望输出:

```

3
2 4 5

```

V03A-EX04 合并果子的堆贪心

- 归属卷: 第 3A 卷
- 覆盖模块: 堆贪心、Huffman 思想、每次取两个最小
- 考场用途: 每次合并产生代价, 总代价最小, 优先考虑小根堆。

题目描述: 有 n 堆果子, 每次可以选择两堆合并, 新堆大小为两堆之和, 本次代价也是两堆之和。求把所有果子合成一堆的最小总代价。

输入格式: 第一行一个整数 n 。第二行 n 个整数表示每堆果子数量。

输出格式: 输出最小总代价。

样例输入:

```

4
1 2 3 4

```

样例输出:

```

19

```

完整代码:

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

```

```

int n;
cin >> n;
priority_queue<long long, vector<long long>, greater<long long>> pq;
for (int i = 1; i <= n; i++) {
    long long x;
    cin >> x;
    pq.push(x);
}

long long ans = 0;
while ((int)pq.size() > 1) {
    long long a = pq.top();
    pq.pop();
    long long b = pq.top();
    pq.pop();
    ans += a + b;
    pq.push(a + b);
}

cout << ans << '\n';
return 0;
}

```

测试设计:

测试 1 输入:

```

4
1 2 3 4

```

期望输出:

```

19

```

测试 2 输入:

```

3
10 20 30

```

期望输出:

```

90

```

V03A-EX05 最少会议室

- 归属卷: 第 3A 卷
- 覆盖模块: 堆贪心、区间扫描、资源复用
- 考场用途: 求最少机器、教室、会议室数量时, 按开始时间扫, 用小根堆维护最早结束资源。

题目描述: 给定 n 个半开区间 $[l, r)$, 同一间会议室中结束时间等于下一场开始时间时可以复用。求最少需要多少间会议室。

输入格式: 第一行一个整数 n 。接下来 n 行, 每行两个整数 $l\ r$ 。

输出格式: 输出最少会议室数量。

样例输入:

```

4
1 4
2 5
6 7
3 8

```

样例输出:

```

3

```

完整代码:

```

#include <bits/stdc++.h>
using namespace std;

```

```

struct Seg {
    long long l;
    long long r;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Seg> seg(n + 1);
    for (int i = 1; i <= n; i++) cin >> seg[i].l >> seg[i].r;

    sort(seg.begin() + 1, seg.end(), [](const Seg &a, const Seg &b) {
        if (a.l != b.l) return a.l < b.l;
        return a.r < b.r;
    });

    priority_queue<long long, vector<long long>, greater<long long>> ends;
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        if (!ends.empty() && ends.top() <= seg[i].l) {
            ends.pop();
        }
        ends.push(seg[i].r);
        ans = max(ans, (int)ends.size());
    }

    cout << ans << '\n';
    return 0;
}

```

测试设计:

测试 1 输入:

```

4
1 4
2 5
6 7
3 8

```

期望输出:

```

3

```

测试 2 输入:

```

5
1 2
2 3
3 4
1 4
4 5

```

期望输出:

```

2

```

V03A-EX06 截止时间课程的反悔贪心

- 归属卷: 第 3A 卷
- 覆盖模块: 反悔贪心、截止时间、最大堆删除最差选择
- 考场用途: 先按截止时间尝试加入, 超时就删掉耗时最长的任务。

题目描述: 有 n 门课程, 第 i 门课程需要 $t[i]$ 天完成, 必须在第 $d[i]$ 天或之前完成。一天只能学习一门课的一天内容。求最多能完成多少门课。

输入格式: 第一行一个整数 n 。接下来 n 行, 每行两个整数 t d 。

输出格式：输出最多能完成的课程数。

样例输入：

```
4
100 200
200 1300
1000 1250
2000 3200
```

样例输出：

```
3
```

完整代码：

```
#include <bits/stdc++.h>
using namespace std;

struct Course {
    long long t;
    long long d;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Course> c(n + 1);
    for (int i = 1; i <= n; i++) cin >> c[i].t >> c[i].d;

    sort(c.begin() + 1, c.end(), [](const Course &a, const Course &b) {
        if (a.d != b.d) return a.d < b.d;
        return a.t < b.t;
    });

    priority_queue<long long> chosen;
    long long total = 0;
    for (int i = 1; i <= n; i++) {
        total += c[i].t;
        chosen.push(c[i].t);
        if (total > c[i].d) {
            total -= chosen.top();
            chosen.pop();
        }
    }

    cout << chosen.size() << '\n';
    return 0;
}
```

测试设计：

测试 1 输入：

```
4
100 200
200 1300
1000 1250
2000 3200
```

期望输出：

```
3
```

测试 2 输入：

3
5 5
4 6
2 6

期望输出:

2

V03A-EX07 0/1 背包反例转 DP

- 归属卷: 第 3A 卷
- 覆盖模块: 贪心反例、性价比失败、0/1 背包 DP
- 考场用途: 看到容量和选或不选时, 不要只按价值密度贪心; 普通 0/1 背包应转 DP。

题目描述: 有 n 个物品, 每个物品有重量 $w[i]$ 和价值 $v[i]$, 每个物品最多选一次。背包容量为 W , 求最大总价值。

输入格式: 第一行两个整数 n W 。接下来 n 行, 每行两个整数 w v 。

输出格式: 输出最大总价值。

样例输入:

3 50
10 60
20 100
30 120

样例输出:

220

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, W;
    cin >> n >> W;
    vector<int> w(n + 1), v(n + 1);
    for (int i = 1; i <= n; i++) cin >> w[i] >> v[i];

    vector<long long> dp(W + 1, 0);
    for (int i = 1; i <= n; i++) {
        for (int cap = W; cap >= w[i]; cap--) {
            dp[cap] = max(dp[cap], dp[cap - w[i]] + v[i]);
        }
    }

    cout << dp[W] << '\n';
    return 0;
}
```

测试设计:

测试 1 输入:

3 50
10 60
20 100
30 120

期望输出:

220

测试 2 输入:

```
4 7
6 13
4 8
3 6
2 4
```

期望输出:

```
14
```

V03A-EX08 普通硬币找零反例转完全背包

- 归属卷: 第 3A 卷
- 覆盖模块: 贪心反例、硬币找零、完全背包 DP
- 考场用途: 币值不是标准币制时, 最大面值优先不可靠, 应使用 DP 求最少硬币数。

题目描述: 给定 n 种硬币面值, 每种硬币可以使用任意多次。求凑出金额 S 所需的最少硬币数; 若无法凑出, 输出 -1 。

输入格式: 第一行两个整数 n S 。第二行 n 个整数表示硬币面值。

输出格式: 输出最少硬币数, 无法凑出输出 -1 。

样例输入:

```
3 6
1 3 4
```

样例输出:

```
2
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, S;
    cin >> n >> S;
    vector<int> coin(n + 1);
    for (int i = 1; i <= n; i++) cin >> coin[i];

    const int INF = 1000000000;
    vector<int> dp(S + 1, INF);
    dp[0] = 0;
    for (int i = 1; i <= n; i++) {
        for (int sum = coin[i]; sum <= S; sum++) {
            dp[sum] = min(dp[sum], dp[sum - coin[i]] + 1);
        }
    }

    cout << (dp[S] == INF ? -1 : dp[S]) << '\n';
    return 0;
}
```

测试设计:

测试 1 输入:

```
3 6
1 3 4
```

期望输出:

```
2
```

测试 2 输入:

2 7
2 4

期望输出:

-1

V03A-EX09 加权区间选择转 DP

- 归属卷: 第 3A 卷
- 覆盖模块: 区间贪心反例、加权区间调度、排序后二分 DP
- 考场用途: 最多活动可以贪心, 但最大收益区间选择通常要 DP。

题目描述: 给定 n 个半开区间 $[l, r)$, 每个区间有收益 w 。选择若干互不重叠区间, 使总收益最大。端点相接不算重叠。

输入格式: 第一行一个整数 n 。接下来 n 行, 每行三个整数 $l\ r\ w$ 。

输出格式: 输出最大总收益。

样例输入:

```
3
1 3 5
3 5 5
1 5 20
```

样例输出:

20

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

struct Job {
    long long l;
    long long r;
    long long w;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Job> job(n + 1);
    for (int i = 1; i <= n; i++) cin >> job[i].l >> job[i].r >> job[i].w;

    sort(job.begin() + 1, job.end(), [](const Job &a, const Job &b) {
        if (a.r != b.r) return a.r < b.r;
        return a.l < b.l;
    }));

    vector<long long> ends(n + 1, 0);
    for (int i = 1; i <= n; i++) ends[i] = job[i].r;

    vector<long long> dp(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        int p = int(upper_bound(ends.begin() + 1, ends.begin() + i, job[i].l) - ends.begin()) - 1;
        dp[i] = max(dp[i - 1], dp[p] + job[i].w);
    }

    cout << dp[n] << '\n';
    return 0;
}
```

测试设计:

测试 1 输入:

```
3
1 3 5
3 5 5
1 5 20
```

期望输出:

20

测试 2 输入:

```
4
1 2 50
2 3 50
1 3 120
3 4 1
```

期望输出:

121

V03A-EX10 相邻限制选择转线性 DP

- 归属卷: 第 3A 卷
- 覆盖模块: 局部最优失败、线性 DP、选或不选
- 考场用途: 相邻不能同时选时, 不要只看当前最大值; 需要记录前缀最优。

题目描述: 给定 n 个位置的收益 $a[i]$, 选择若干位置, 使任意两个被选位置不相邻, 最大化收益和。

输入格式: 第一行一个整数 n 。第二行 n 个整数。

输出格式: 输出最大收益和。

样例输入:

```
5
2 7 9 3 1
```

样例输出:

12

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> a(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> a[i];

    vector<long long> dp(n + 1, 0);
    if (n >= 1) dp[1] = max(0LL, a[1]);
    for (int i = 2; i <= n; i++) {
        dp[i] = max(dp[i - 1], dp[i - 2] + a[i]);
    }

    cout << dp[n] << '\n';
    return 0;
}
```

测试设计:

测试 1 输入:

5
2 7 9 3 1

期望输出:

12

测试 2 输入:

4
5 1 1 5

期望输出:

10

V03A-CEX01 活动选择结束时间贪心

- 归属卷: 第 3A 卷
- 覆盖模块: 区间贪心、交换论证
- 考场用途: 最经典可贪心模型。
- 参考题型来源: 参考来源: 洛谷区间贪心题型。

题目描述: 给出若干活动起止时间, 求最多选多少个互不重叠活动。

输入格式: 第一行 n , 之后 n 行 $l\ r$ 。

输出格式: 输出最多数量。

样例输入:

4
1 3
2 4
3 5
6 7

样例输出:

3

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<int,int>> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i].second >> a[i].first;
    sort(a.begin() + 1, a.end());
    int cnt = 0, last = -2000000000;
    for (int i = 1; i <= n; i++) if (a[i].second >= last) {
        cnt++;
        last = a[i].first;
    }
    cout << cnt << '\n';
    return 0;
}
```

测试设计: 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

V03A-CEX02 会议室最少数量

- 归属卷: 第 3A 卷
- 覆盖模块: 排序、堆贪心

- 考场用途：同时占用区间数量用小根堆。
- 参考题型来源：参考来源：经典会议室/区间调度题型。

题目描述：给出会议时间，求最少会议室数量。

输入格式：第一行 n ，之后 n 行 $l\ r$ 。

输出格式：输出最少数量。

样例输入：

```
3
0 30
5 10
15 20
```

样例输出：

```
2
```

完整代码：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<int,int>> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i].first >> a[i].second;
    sort(a.begin() + 1, a.end());
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int i = 1; i <= n; i++) {
        if (!pq.empty() && pq.top() <= a[i].first) pq.pop();
        pq.push(a[i].second);
    }
    cout << pq.size() << '\n';
    return 0;
}
```

测试设计：额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

V03A-CEX03 最少点刺破区间

- 归属卷：第 3A 卷
- 覆盖模块：区间按右端排序
- 考场用途：证明每次选最早结束点。
- 参考题型来源：参考来源：洛谷区间选点题型。

题目描述：选择尽量少的点，使每个闭区间至少包含一个点。

输入格式：第一行 n ，之后 n 行 $l\ r$ 。

输出格式：输出最少点数。

样例输入：

```
3
1 3
2 5
6 8
```

样例输出：

```
2
```

完整代码：

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<int,int>> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i].first >> a[i].second;
    sort(a.begin() + 1, a.end(), [](auto x, auto y) {
        return x.second < y.second;
    });
    int points = 0, last = -2000000000;
    for (int i = 1; i <= n; i++) {
        if (last < a[i].first) {
            last = a[i].second;
            points++;
        }
    }
    cout << points << '\n';
    return 0;
}

```

测试设计：额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

V03A-CEX04 单价贪心失败的 0/1 背包

- 归属卷：第 3A 卷
- 覆盖模块：反例、0/1 背包
- 考场用途：看到选/不选和容量，别按性价比硬贪。
- 参考题型来源：参考来源：洛谷背包题单。

题目描述：给 0/1 背包，输出最大价值。

输入格式：第一行 n W ，之后 w v 。

输出格式：输出最大价值。

样例输入：

```

3 50
10 60
20 100
30 120

```

样例输出：

```

220

```

完整代码：

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, W;
    cin >> n >> W;
    vector<int> w(n + 1), v(n + 1);
    for (int i = 1; i <= n; i++) cin >> w[i] >> v[i];
    vector<int> dp(W + 1, 0);
    for (int i = 1; i <= n; i++) {
        for (int j = W; j >= w[i]; j--) dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}

```

```

    }
    cout << dp[W] << '\n';
    return 0;
}

```

测试设计: 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

V03A-CEX05 截止日任务反悔贪心

- 归属卷: 第 3A 卷
- 覆盖模块: 排序、堆、反悔贪心
- 考场用途: 先收下, 超过容量就反悔删最小收益。
- 参考题型来源: 参考来源: 经典课程安排/反悔贪心题型。

题目描述: 每个任务有截止日和收益, 每天最多做一个, 求最大收益。

输入格式: 第一行 n T , 之后 deadline profit 。

输出格式: 输出最大收益。

样例输入:

```

4 4
1 10
2 20
2 15
3 5

```

样例输出:

```

40

```

完整代码:

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, T;
    cin >> n >> T;
    vector<pair<int, int>> jobs(n + 1);
    for (int i = 1; i <= n; i++) cin >> jobs[i].first >> jobs[i].second;
    sort(jobs.begin() + 1, jobs.end());
    priority_queue<int, vector<int>, greater<int>> pq;
    long long sum = 0;
    for (int i = 1; i <= n; i++) {
        pq.push(jobs[i].second);
        sum += jobs[i].second;
        if ((int)pq.size() > jobs[i].first) {
            sum -= pq.top();
            pq.pop();
        }
    }
    cout << sum << '\n';
    return 0;
}

```

测试设计: 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。
